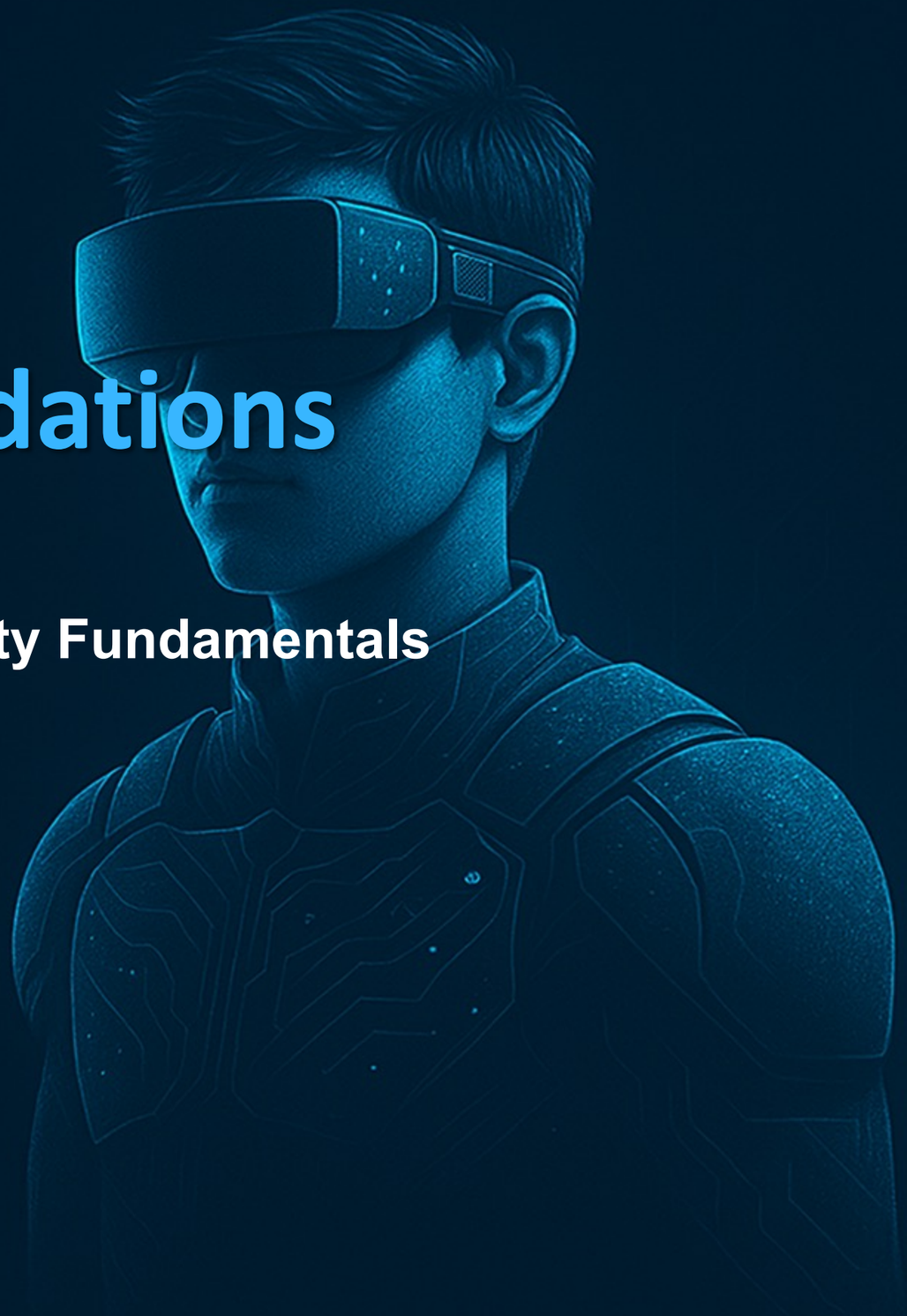
CYBERLABS

Linux Foundations

Module 6 — Cybersecurity Fundamentals



CYBERLABS

Today

- Linux lecture — what it is and how to use it (60 min)
- Bash scripting — automate the boring parts (30 min)
- Linux lab — hands-on with WSL2, PuTTY and WinSCP (30 min)
- Break (15 min)
- Docker — containers for the Day 5 labs (afternoon)
- Goal: survive every lab that needs a Linux command line

CYBERLABS

What is Linux

Kernel and operating system

- An operating system runs programs and manages hardware
- The kernel is the core that talks to CPU, memory and disks
- Linux is the kernel; add tools around it and you get an OS
- Free, open source, and runs most servers on the internet
- Created in 1991 by Linus Torvalds, still actively developed
- Stable, scriptable, and built for remote control

Linux vs Windows

- Windows leads on desktops; Linux leads on servers
- Linux is driven by the command line, not just clicks
- Free to install and copy — no license keys
- You can see and change the source code
- Different file layout: no C: drive, paths use /
- Most security tooling is built for Linux first

CYBERLABS

Distributions (distros)

- A distro = the Linux kernel plus a chosen set of tools
- Kali Linux — packed with offensive security tools (red team)
- Ubuntu — popular for servers and defenders (blue team)
- Both are Debian-family, so they share the apt installer
  - Other families exist (Fedora, Arch) — different tools
- We target Kali and Ubuntu all day

CYBERLABS

Where security pros meet Linux

- Servers and cloud hosts you defend or attack
- Pen-testing platforms like Kali
- Security tools shipped as Linux containers (Splunk, Volatility)
- Log files — almost all live on Linux systems
- Remote boxes reached over SSH with no screen attached
- Bottom line: comfort here is a job requirement

CYBERLABS

Shell & Getting In

CYBERLABS

Terminal vs shell

- Terminal — the window where you type commands
- Shell — the program inside that reads and runs your commands
- Bash is the most common shell; it is our default
- You type a line, press Enter, the shell runs it
- `Prompt ends in $ for a normal user`
- `Prompt ends in # when you are root (admin)`

CYBERLABS

WSL2 — Linux on Windows

- WSL2 = Windows Subsystem for Linux, version 2
- Runs a real Linux kernel inside Windows — no full VM
- Lets you use Linux without leaving your Windows machine
- Already enabled on your lab computer
- `Open it by running: wsl`
- We use it for the hands-on Linux lab today

Anatomy of a command

- Pattern: command -flags arguments
- Command — the program to run
- Flags (options) — change behaviour, usually start with -
- Arguments — what the command acts on, like a filename
- `ls -l /etc`
  - ls = command, -l = flag, /etc = argument

Getting help

- Never memorise everything — look it up
- `man ls`
  - Opens the manual page; press q to quit
- `ls --help`
  - Quick summary printed right in the terminal
- When stuck: read the man page first, then search online

CYBERLABS

Filesystem & Navigation

Everything is a file

- In Linux almost everything is treated as a file
- Documents and folders — files
- Devices like disks and USB sticks — files
- Even running system info appears as files
- One simple model: learn file commands, control the system
- Folders are called directories in Linux

CYBERLABS

The filesystem layout (FHS)

- / – the root, the top of everything
- /etc – system configuration files
- /var/log – log files (key for security work)
- /home – personal folders for each user
- /tmp – temporary scratch space, wiped on reboot
- /bin – common programs and commands

CYBERLABS

Absolute vs relative paths

- A path is the address of a file or directory
- `Absolute path starts at root: /home/student/notes.txt`
- Works from anywhere — it is the full address
- `Relative path starts from where you are: notes.txt`
- `. means here, .. means the parent directory`
- Use absolute when in doubt — no ambiguity

Where am I? pwd and cd

- `pwd – print working directory (where you are now)`
- `cd /var/log – change directory to /var/log`
- `cd .. – move up one level`
- `cd ~ – go to your home folder`
- `cd with no argument also goes home`
- Lost? Run pwd to get your bearings

CYBERLABS

Listing files with ls

- `ls` – list files in the current directory
- `ls -l` – long format: permissions, owner, size, date
- `ls -a` – also show hidden files (names start with .)
- `ls -h` – human-readable sizes (KB, MB) with -l
- `ls -lah` – combine flags into one
- Hidden files are often config files worth a look

CYBERLABS

Files & Text

Creating files and folders

- `touch report.txt` — create an empty file
- `mkdir evidence` — create a new directory
- `mkdir -p case/2026/may` — create nested directories
- touch also updates the timestamp of an existing file
- Names with spaces need quotes or are awkward — avoid spaces
- Confirm with ls afterwards

CYBERLABS

Copy, move, rename

- `cp file.txt backup.txt` – copy a file
- `cp -r folder1 folder2` – copy a whole folder
- `mv file.txt /tmp/` – move a file elsewhere
- `mv old.txt new.txt` – rename a file
- There is no separate rename command — mv does both
- Moving and renaming are the same operation in Linux

Deleting — and the rm danger

- `rm file.txt – delete a file`
- `rm -r folder – delete a folder and its contents`
- There is no recycle bin — rm is permanent
- `Never run: rm -rf /`
  - It would try to erase the entire system
- Read the rm command twice before pressing Enter

CYBERLABS

Viewing file contents

- `cat notes.txt` — dump the whole file to the screen
- `less notes.txt` — scroll a big file; q to quit
- `head -n 20 log.txt` — show the first 20 lines
- `tail -n 20 log.txt` — show the last 20 lines
- `tail -f log.txt` — follow a log live as it grows
- Use less for big files; cat for short ones

CYBERLABS

Finding files with find

- `find searches directories for files by name or type`
- `find /home -name notes.txt – find by exact name`
- `find / -name "*.log" – find all .log files`
- `find /var -type d – find only directories`
- First argument = where to start searching
- Great for locating evidence or stray files

CYBERLABS

Searching inside files with grep

- `grep` searches for text inside files
- `grep "error" log.txt` – show lines containing error
- `grep -i "error" log.txt` – ignore upper/lowercase
- `grep -r "password" /etc` – search a whole folder
- `grep -n "error" log.txt` – show line numbers
- A core skill for hunting through logs

CYBERLABS

Pipes and redirection

- A pipe | sends one command's output into another
- cat log.txt | grep "error" – search the file output
- > writes output to a file (overwrites it)
- grep "error" log.txt > errors.txt
- >> appends output instead of overwriting
- Combine small tools to build powerful one-liners

CYBERLABS

Permissions & Users

CYBERLABS

Users, groups, and root

- Every file is owned by a user and a group
- A user is an account; a group bundles users together
- root is the all-powerful administrator account
- Normal users are limited — that is a safety feature
- `whoami – show which user you are`
- `id – show your user and group memberships`

Reading rwx permissions

- `ls -l shows permissions like: -rwxr-xr--`
- r = read, w = write, x = execute (run)
- Three groups: owner, group, others (everyone else)
  - rwx = owner, r-x = group, r-- = others
- A dash means that permission is missing
- First character: - is a file, d is a directory

CYBERLABS

Changing permissions with chmod

- `chmod changes who can read, write or run a file`
- `chmod +x script.sh – make a file executable`
- `chmod -w file.txt – remove write permission`
- `chmod 644 file.txt – owner read/write, others read`
- `chmod 755 script.sh – owner all, others read/run`
- Numbers: 4 read, 2 write, 1 execute, added together

CYBERLABS

Changing ownership with chown

- `chown changes who owns a file`
- `chown student file.txt – set the owner to student`
- `chown student:staff file.txt – set owner and group`
- `chown -R student folder – apply to a whole folder`
- Usually needs sudo because it changes ownership
- Often paired with chmod when fixing access problems

CYBERLABS

sudo and least privilege

- `sudo runs one command as root, the administrator`
- `sudo apt update – run an admin task safely`
- Least privilege — use admin power only when needed
- Working as root all day risks costly mistakes
  - SUID — a file that runs as its owner, not you
  - Attackers hunt misconfigured SUID files to gain root

CYBERLABS

Processes & Packages

CYBERLABS

Processes — ps, top, kill

- A process is a running program
- `ps aux` — list all running processes
- `top` — live, updating view of activity; q to quit
- Each process has a PID, a unique ID number
- `kill 1234` — stop the process with PID 1234
- `kill -9 1234` — force-stop a stuck process

CYBERLABS

Services with systemctl

- A service is a program that runs in the background
  - Examples: web servers, SSH, logging
- `systemctl status ssh` – check if a service is running
- `sudo systemctl start ssh` – start a service
- `sudo systemctl stop ssh` – stop a service
- `sudo systemctl enable ssh` – start it at every boot

Installing software with apt

- apt is the package manager on Kali and Ubuntu
- It downloads and installs programs for you
- `sudo apt update – refresh the list of available software`
- `sudo apt install nmap – install a program`
- `sudo apt remove nmap – uninstall a program`
- Always run apt update before installing

CYBERLABS

Command cheat sheet

- pwd ls cd – where am I, what's here, move around
- touch mkdir cp mv rm – create, copy, move, delete
- cat less head tail – view file contents
- find grep – locate files, search inside files
- chmod chown sudo – permissions and admin power
- ps top kill systemctl apt – processes and software

CYBERLABS

Bash Scripting

CYBERLABS

Why automate?

- Typing the same commands repeatedly is slow and error-prone
- A script runs a saved list of commands in order
- Write it once, run it perfectly every time
- Great for repetitive checks, log gathering, setup tasks
- Security work is full of repeatable routines
- Bash scripting turns tedious jobs into one command

CYBERLABS

What is a script? The shebang

- A script is a plain text file full of commands
- The first line tells the system which shell to use
- `#!/bin/bash`
  - This first line is called the shebang
- `Lines starting with # are comments – ignored`
- Save scripts with a .sh ending by convention

Make it executable and run it

- A new script file cannot run until you allow it
- `chmod +x script.sh – mark it as executable`
- `./script.sh – run a script in the current folder`
  - The ./ means 'in this directory'
- `bash script.sh – run it without chmod`
- Forgetting chmod +x is the most common beginner error

Variables

- A variable stores a value you can reuse
- `name="Alice" – assign a value (no spaces around =)`
- `echo "$name" – use the value, prefix with $`
- Set without $, read with $
- `count=5`
- `echo "There are $count files"`

CYBERLABS

Quoting — double vs single

- `Double quotes "..." – variables are expanded`
- `echo "Hello $name"  →  Hello Alice`
- `Single quotes '...' – text is literal, no expansion`
- `echo 'Hello $name'  →  Hello $name`
- Always quote variables to be safe with spaces
- When unsure, use double quotes

CYBERLABS

Reading input with read

- `read pauses the script and waits for the user to type`
- `read name – store what the user types in name`
- `echo "Enter your name:"`
- `read name`
- `echo "Hello $name"`
- Makes scripts interactive instead of fixed

Making decisions — if and test

- if runs commands only when a condition is true
- Square brackets [...] perform the test
- if ["$count" -gt 3]; then
-   echo "More than three"
- fi – closes the if block
- -eq equal, -gt greater, -lt less; -f file exists

Repeating — for loops

- A for loop repeats commands for each item in a list
- for file in *.log; do
- echo "Found: $file"
- done – closes the loop
- for n in 1 2 3; do echo "$n"; done
- Loops process many files without copy-paste

Worked example — log checker (1/2)

- `#!/bin/bash`
- `# Count error lines in each log file`
- `echo "Which folder holds the logs?"`
- `read folder`
  - Asks the user where the logs are
- Continues on the next slide

Worked example — log checker (2/2)

- `for file in "$folder"/*.log; do`
- `  count=$(grep -c "error" "$file")`
- `  echo "$file has $count errors"`
- `done`
  - Loops every .log file and counts error lines
- `Run it: chmod +x check.sh  then  ./check.sh`

CYBERLABS

Common pitfalls

- `Spaces around = break assignment: name = "x" fails`
- `Forgetting chmod +x – the script will not run`
- Unquoted variables break on spaces in names
- `Missing spaces inside [ ] – the test fails`
- Forgetting fi or done leaves a block unclosed
- Test on safe files before running anything destructive

Summary

- Linux is the OS behind servers and security tools
- The shell runs commands: command -flags arguments
- Navigate and manage files with pwd, cd, ls, cp, mv, rm
- Search with find and grep; chain tools with pipes
- Permissions (rwx, chmod) and sudo control access
- Bash scripts automate repeatable tasks — write once, run often

CYBERLABS

Thank you